In Charge

Build plan: from nothing to submission

Meta Global AI Developer Hackathon 2026 - Indie Developer Track - written Nov 1, 2026

Key dates (all IST)	
Now	Sun Nov 1, evening. You start here.
Code freeze	Wed Nov 4, 6:00 PM
Your hard deadline: submit before	Wed Nov 4, 10:00 PM
Official cutoff	9:00 PM ET on Nov 4 = 7:30 AM IST on Nov 5. Do not use this buffer.
Judging / winners	Judging Nov 4 to 8. Closing ceremony and winners Nov 9.

1. The idea in plain words

Problem. You ship a project and move on. Weeks later a dependency update breaks the build, an API changes its response, or a user reports a bug, and nobody notices until someone clicks your demo and sees an error page. Students, freelancers, small teams and open-source maintainers all live with this.

One-sentence pitch. In Charge is an AI maintainer that keeps a shipped project alive: it watches GitHub, the running app and the owner's chat, proves each bug with a failing test, fixes it in a sandbox, and opens a pull request with evidence, under rules the owner sets.

Why it fits the judging. The Indie track is judged on creativity, technical ambition and whether it works. Creativity: an agent that owns operations, not just code, with an owner-written charter. Ambition: a multi-agent loop with a sandbox and a reproduce-first gate. Works: a measured eval on 10 failure scenarios. Theme: Push the Frontier (software).

Honest note. Coding agents are a crowded space (GitHub already ships one). Your edge is the operational angle, the charter, measured results, and a clean demo. Nobody can promise a top-5 finish; what you control is a demo that visibly works and numbers you can defend.

2. Decisions already made (do not reopen)

Topic	Choice	Why
Form factor	Backend service plus web dashboard	It must run 24/7 and react to events. A browser extension cannot, and a mobile app is wasted time.
Language / stack	Python FastAPI backend, React dashboard, Postgres (Neon or Supabase free)	Fast to build with AI agents. No Rust: it adds compile-error loops and nothing in this project needs it.
GitHub link	Fine-grained personal access token plus webhooks, on one demo repo	A full GitHub App is cleaner but costs a day. Say 'GitHub App ready' under roadmap only.
Sandbox	Modal sandboxes (fallback: Docker on your laptop)	Runs untrusted code safely, uses your partner credits. Hide it behind one interface so you can swap.
Models	muse-wonder-1.0 for the main In Charge agent; muse-spark-1.3 for sub-agents	Via Meta Model API, OpenAI-compatible, base URL https://api.meta.ai/v1. Tool calling must be verified at Gate 0.
Koda	Optional fixer engine, only if the 30-minute test passes	MIT licensed Rust coding agent. Keep its LICENSE and NOTICE, credit the author, never rebrand it. Your original work is the In Charge layer.

Topic	Choice	Why
Scope	One demo repo, no login, no multi-user, no real cloud deploy	A merged PR with green CI is the finish line for a fix.

3. How it works

One incident, end to end

- [ ] **Event arrives** from one of four sources: a GitHub webhook (new issue, comment, failed CI run), a health poller hitting the deployed demo app every 60 seconds, an error report posted by the app, or the owner's message in the dashboard chat.
- [ ] **Incident created** with the evidence attached.
- [ ] **In Charge** (muse-wonder-1.0) loads the charter and project memory, then decides: ignore, ask the owner, or fix.
- [ ] **Triage** classifies, sets severity, and de-duplicates against open incidents.
- [ ] **Reproducer** writes a failing test in the sandbox. If it cannot reproduce, it comments asking for details and stops. It never guesses a fix.
- [ ] **Fixer** patches the code (Koda headless or your own loop). The sandbox runs the full test suite.
- [ ] **Verifier** checks the diff against the charter: size limit, allowed paths, no secrets touched.
- [ ] **PR opened** with the failing test, the fix, logs and a plain-language explanation. The charter decides whether it may auto-merge once CI is green.
- [ ] **Recovery confirmed** by the health check. Incident closed, the owner gets a short report, and a lesson is written to project memory.

Components

Component	Job	Model / tech
Watchers	Turn GitHub, health and chat events into incidents	No model; FastAPI, webhooks, poller
In Charge agent	Decide, delegate, approve, report to owner	muse-wonder-1.0
Triage and Reporter	Classify, de-duplicate, write PR text and messages	muse-spark-1.3
Reproducer, Fixer, Verifier	Write failing test, patch, check the diff	muse-spark-1.3 (Wonder for hard cases); Koda optional
Sandbox runner	Clone at a commit, install, run tests, return the diff	Modal sandbox
Memory	Project journal: past incidents, fragile areas, owner decisions	Postgres table. Use Wonder's memory features only if the API exposes them.
Dashboard	Incident feed, live agent trace, chat, charter editor, metrics	React plus server-sent events

4. Repo layout and the charter

```
in-charge/
  AGENTS.md                rules every agent must follow
  contract/
    incident.schema.json   the Incident record
    events.md              webhook and health event shapes
    api.md                 REST and SSE endpoints the dashboard uses
    tools.md               tool definitions the agents may call
    charter.schema.json    charter file format
    fixtures/              sample events, incidents, diffs
  backend/app/
    github/                webhooks, GitHub client, PR ops          (feat/github)
    sandbox/               Modal runner, reproduce-fix-verify loop  (feat/sandbox)
    brain/                 agents, prompts, memory, charter parser  (feat/brain)
    api/                   FastAPI routes, SSE stream
  dashboard/               React UI                                 (feat/dashboard)
  eval/                    scenarios, runner, results.json
  README.md
```

Example charter (lives in the target repo as .incharge/charter.yml)

```
version: 1
repo: yourname/incharge-demo-app
autonomy:
  auto_fix: [failing_tests, patch_dependency_bump, docs, small_bugs]
  ask_first: [auth, database_schema, payments, deleting_files]
  never: [push_to_main, change_secrets]
limits:
  max_files_changed: 5
  max_lines_changed: 150
  max_cost_usd_per_incident: 1.50
  allowed_paths: ["app/**", "tests/**", "requirements.txt"]
  forbidden_paths: [".github/workflows/**", ".env*"]
merge:
  auto_merge: false          # true only in away mode, and only if CI is green
away_mode:
  enabled: false
notify:
  channel: dashboard_chat
```

5. Step-by-step plan (times in IST)

Phase 0 - Sunday Nov 1, now to about 8 PM: setup and go/no-go

- [ ] **0.1 Read the Official Rules and the submission form** (15 min). Look for: rules on pre-existing code or open-source use, video length limit, whether the repo must be public, required fields, how the track and theme are chosen.
- [ ] **0.2 Accounts and repos** (20 min). GitHub: create *in-charge* (public, MIT) and *incharge-demo-app* (public). Also create Modal, Neon or Supabase (Postgres), and Render accounts.
- [ ] **0.3 Gate 0 - Meta API** (30 min). Create a key at dev.meta.ai. With an OpenAI-compatible client and base URL https://api.meta.ai/v1, make one call to muse-wonder-1.0 and one to muse-spark-1.3, each with a tool-calling request. Write down: works or not, latency, errors. If it fails, ask in the hackathon Discord office hours right away and use partner-hosted open weights (Baseten or Fireworks) so you are not blocked.
- [ ] **0.4 Gate 1 - Koda test** (30 min, hard timebox). Install Koda (Homebrew or a prebuilt binary; building from source can be slow). Point it at the Meta endpoint as a custom OpenAI-compatible provider. Run one headless task on a toy repo (for example 'fix the failing test'). PASS: it edits code and the test goes green. FAIL or unclear after 30 minutes: drop Koda and write a simple fixer loop yourself (read files, propose a patch with tool calls, run tests). Both paths are fine. Do not spend more time.
- [ ] **0.5 Record the results** at the top of the README as a short table: verified versus assumed.

Phase 1 - Sunday Nov 1, about 8 PM to midnight: contract and launch

- [ ] **1.1 One agent builds the scaffold on main**: the repo layout above, AGENTS.md, contract files, fixtures, .env.example, a one-command run script. Mock first: the backend returns fixture data so every branch runs alone.
- [ ] **1.2 You review the contract and change it now**, not later. Merge to main and tag v0.
- [ ] **1.3 In parallel, a second agent builds the demo app**: a small FastAPI app with a real pytest suite in *incharge-demo-app*, deployed to Render.
- [ ] **1.4 Launch four agents** on four branches (Section 6), each in its own git worktree, each given only AGENTS.md, contract/ and its own brief. Do not paste this PDF to them.

Done when: four agents are running and the demo app is live with green tests.

Phase 2 - Monday Nov 2: first full path

- [ ] Review and merge agent PRs within one to two hours of each arriving. Small PRs, merged often.
- [ ] **By about 2 PM:** real GitHub events become incidents and appear in the dashboard feed (no fixes needed yet).
- [ ] **By about 8 PM, Gate 2:** scenario 1 end to end. You open an issue on the demo repo; In Charge reproduces the bug, fixes it, opens a PR; the dashboard shows the live trace.
- [ ] If Gate 2 is not met by 8 PM, cut scope using Section 11. Do not add features.
- [ ] Evening: write break scripts for scenarios 2 and 3.

Phase 3 - Tuesday Nov 3: scale and measure

- [ ] Morning: scenarios 4 to 10 as break scripts; charter enforcement; memory journal; owner chat.
- [ ] Midday: run the eval on all 10 scenarios. Fix the top two failure causes and rerun once. Keep the honest numbers.
- [ ] Afternoon: record the demo video (two or three takes, Section 9).
- [ ] Evening: README draft with architecture, eval table, limits and Koda attribution; second eval run; save results.json.
- [ ] **Gate 3 by midnight:** measured numbers and a recorded demo exist. Everything after this is polish.

Phase 4 - Wednesday Nov 4: polish and submit

[ ] Morning: retake weak demo segments; finish the README; test a fresh clone by following the README exactly.

[ ] **6:00 PM: freeze.** Tag v1.0. No more code changes.

[ ] 6 to 9 PM: submit (form, repo link, video link). Open every link in a private window to confirm it works.

[ ] **10:00 PM: hard stop.** Sleep.

6. Working with parallel agents

Rules every agent follows (put these in AGENTS.md)

- You own only your directory. Do not edit another agent's files.
- Changes to contract/ go through a PR that touches contract/ alone, approved by the human.
- Mock first: your code must run alone using fixtures.
- Small PRs (under about 400 lines) with tests, merged at least twice a day. No secrets in git; use .env.
- Log every external call and agent decision (audit trail). Never write to a default branch.
- Blocked for more than 20 minutes: write BLOCKERS.md and tell the human.

The four briefs (give each agent only its own)

Branch	Builds	Done when
feat/github	Webhook receiver with signature check (issues opened or commented, workflow run failed); GitHub client (branch, commit patch, open PR, comment, label); incident creation; health poller; error-ingest endpoint.	A real issue on the demo repo becomes an Incident within 5 seconds. A PR can be opened from a diff fixture.
feat/sandbox	Modal runner (clone at a commit, install, run tests, return results); reproduce step; fix step (Koda headless or own loop); verify step (full tests plus charter diff checks).	Given an Incident fixture and the demo repo, it returns a verified diff for scenario 1.
feat/brain	In Charge tool-calling loop (muse-wonder-1.0); sub-agents (muse-spark-1.3); tools per contract; charter parser and enforcer; memory journal read and write; cost cap; decisions logged.	From an Incident fixture it plans and calls the sandbox and GitHub tools (mocked), asks when the charter says ask, and stops when it cannot reproduce.
feat/dashboard	React with server-sent events: incident feed, live agent trace, PR links, chat box, charter view and edit, metrics page (fix rate, recovery time).	It renders from fixtures, then works against the live backend with no code change.

You do: the demo app (with your agent), the scenarios and break scripts, the eval runner, the README and the demo video. You are also the integrator: merge, run the eval, and keep one running log of measured numbers.

7. Demo app, scenarios and the eval

Build each scenario as a script that breaks the demo app in one specific way, so demos and evals are repeatable. Keep a held-out set of tests the agent never sees; use it to grade fixes.

#	How it breaks	Source	Right outcome
1	Off-by-one in pagination	User issue	Reproduce, fix, PR
2	Dependency version bump breaks an import	Failing CI	Fix or pin, PR
3	Upstream API renames a field, causing a 500	Error report / health	Fix the parser, PR
4	Refactor leaves a failing unit test	Failing CI	Fix, PR

#	How it breaks	Source	Right outcome
5	Missing environment variable crashes startup	Health check	Escalate to owner (a secret) and PR a clearer error
6	Timezone bug reported in chat	Owner chat	Reproduce, fix, PR
7	Empty input causes a 500	User issue	Fix, PR
8	A route returns 404 after a change	Health check	Fix, PR
9	Security advisory needs a patch-level bump	Dependency alert	Bump, run tests, PR
10	Vague report: 'it is slow sometimes'	User issue	Cannot reproduce: ask for details, open NO PR

Metrics to report

Metric	Definition
Fix rate	Scenarios 1 to 9 with a verified fix PR, divided by 9
Wrong-PR rate	PRs where held-out tests fail or forbidden files changed
Escalation correctness	Scenarios 5 and 10 handled by asking, not guessing
Time to recovery	Median time from event to PR opened (and to health green when auto-merged)
Cost per incident	Token cost per incident from usage logs

Run each scenario at least twice and report both runs. A measured, modest result you can defend beats a high number you cannot.

8. Safety rules (these are also demo talking points)

- Never push to the default branch. PRs only.
- Path allowlist and forbidden paths from the charter; diff size caps; no changes to CI workflows or secrets.
- No secrets inside the sandbox. A spending cap per incident. A kill switch in the dashboard.
- Reproduce before fixing. If it cannot reproduce, it asks and stops.
- Every action and decision goes to the audit log shown in the dashboard.

9. Demo video script (about 3 minutes; confirm the limit in the rules)

Time	Show	Say
0:00 - 0:20	A broken demo link	Every project you ship starts rotting the day you stop looking.
0:20 - 0:50	Dashboard and charter	In Charge is an AI maintainer with rules the owner writes.
0:50 - 1:50	Live: user files an issue, the agent trace, failing test, fix, real PR page	It proves the bug first, then fixes it, and shows its evidence.
1:50 - 2:20	CI break caught; scenario 10 where it asks instead of guessing	It knows when not to act.
2:20 - 2:45	Eval table	Measured on 10 failures: fix rate, wrong-PR rate, recovery time.

Time	Show	Say
2:45 - 3:00	Architecture slide	Built with Muse Wonder and Muse Spark (and Koda as the fixer engine, if used). Next: GitHub App, more repos.

Tips: pre-warm everything before recording, record two or three takes, show the real GitHub PR page, add subtitles.

10. Submission checklist

[ ] Public repo with README: what it is, why, architecture diagram, how to run, eval table, honest limits.

[ ] Clear credit: what is Koda (if used), what you built, which Meta models you used and for what.

[ ] License file in your repo; Koda's LICENSE and NOTICE kept if you include any of its code.

[ ] Demo video link that opens in a private window; repo link tested the same way.

[ ] Track: Indie Developer. Theme: Push the Frontier. One-line pitch from Section 1 at the top of the README.

[ ] A fresh-clone test done by following the README exactly.

[ ] Everything submitted before Nov 4, 10:00 PM IST.

11. Risks, fallbacks and the cut order

Risk	Likelihood	Fallback
Fixes fail on real bugs	High	Choose solvable scenarios, reproduce first, report honest numbers
Meta API access, region or limits	Medium	Gate 0 tonight; Discord office hours; open weights on Baseten or Fireworks
Modal sandbox setup is slow	Medium	Local Docker behind the same interface
Koda does not fit	Medium	Decided at Gate 1: own fixer loop
Agents conflict or drift	Medium	Directory ownership, frozen contract, merge twice a day
Scope creep	Certain	Use the cut order below
No sleep	High	Freeze at 6 PM Nov 4; at least 6 hours of sleep each night

Cut order if you are behind: dashboard polish, then away mode, then auto-merge, then memory beyond a simple journal, then the health poller (keep the issue and CI path), then reduce scenarios from 10 to 6.

Never cut: the reproduce-first gate, the eval numbers, the demo video.

12. Daily rhythm

- Start of day: merge open PRs, run the eval, write the numbers in one log file.
- During the day: one question at a time to your agents, small tasks, commit often.
- End of day: tag a working version, so you can always roll back to something that works.
- Free-quota tip: give each agent only AGENTS.md, contract/ and its own brief. Do not paste long documents.

Facts to verify at Gate 0: the Meta API base URL, model IDs and tool-calling support come from public documentation and coverage, not from your account. Treat them as assumed until your own test succeeds.